Solving Chess Endgames

A Reinforcement Learning Approach

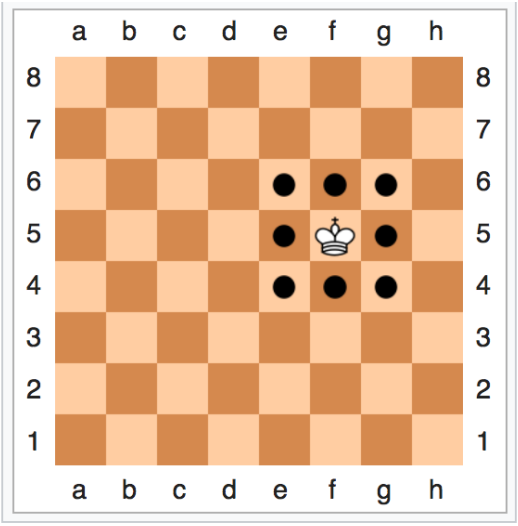
Valeria De Stasio, Christian Faccio, Giovanni Lucarelli

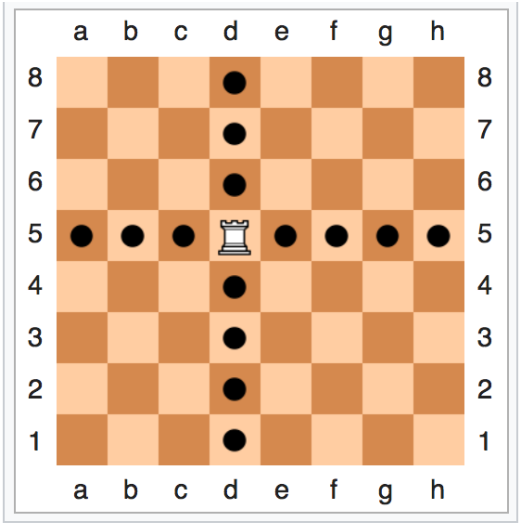
September 5, 2025

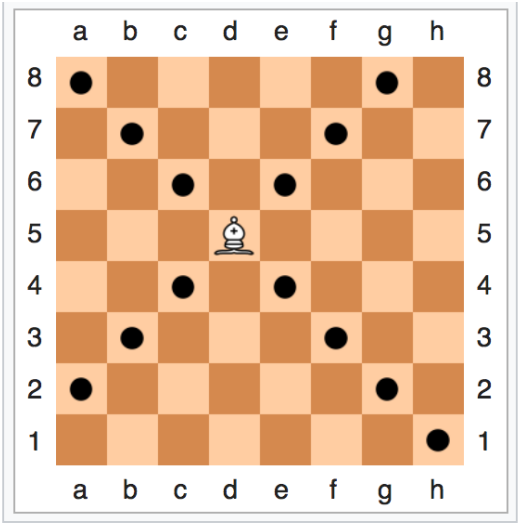
Chess Programming Background

- 1890 El Ajedrecista: electro-mechanical KRK solver
- 1950 Claude Shannon: first article about chess programming
- 1997 *Deep Blue*: first computer program to beat the current world champion
- 2008 *Stockfish*: strong heuristic-based chess engine
- 2017 *Alphazero*: neural network-based approach that defeated Stockfish
- from 2018: more models developed based on *Alphazero*

Chess Rules







- **Win**: checkmate the opponent's king
- **Draw**: stalemate or insufficient material[1]

[1] threefold repetition and the fifty-move rule are not considered in our implementation

Elementary Endgames

- subset of endgames with few pieces left
- clear theoretical results: forced checkmate or draw
- foundation for learning more complex endgames
- examples: KRK, KQK, KBBK
- solved with tablebases up to 7 pieces

MDP Formulation

Markov Game vs MDP

- Chess is a 2-player, turn-based, zero-sum game, formally a Markov Game
- Assume fixed black defender policy: **oracle** or **random**
- Black becomes part of the environment: Markov Game $\rightarrow$ MDP

Remark: white is always winning in elementary endgames!

MDP: States

- State space $\mathcal{S}$: legal pieces positions and side-to-move

```
8/8/K7/8/8/5k2/7R/8 w - - 0 1
```

```
std::array<std::array<U64, 6>, 2> pieces;
Color side_to_move;
```

- Terminal state space $\bar{\mathcal{S}} \subset \mathcal{S}$: checkmate or draw positions (and stm). Defined procedurally

```
state.is_game_over() -> bool
```

MDP: Actions

- Action space $\mathcal{A}(s)$: set of legal moves for the current player.
- Defined procedurally:

```
state.legal_moves() -> std::vector<Move>
```

MDP: Transition

Transition probability $\mathbf{Pr}(S_{t+1} = s' | S_t = s, A_t = a)$

- **Oracle Defender**: deterministic evolution

```
state.do_move(Move)                 // white deterministic
state.do_move(oracle.reply())       // black deterministic
```

- **Random Defender**: stochastic evolution

```
state.do_move(Move)                   // white deterministic
state.do_move(random_reply())         // black random
```

MDP: Reward

Goal: find the fastest mate

Reward:

- checkmate: +1
- draw: -1 (always winning for white)
- encourage faster mates
  - step penalty: -2
  - discount factor $\gamma = 0.99$

RL Algorithms

Results

Assessment Metrics Overview

* DTM, DTZ etc * success, top1 etc

Policy interpretability through human chess principles

Video Animation of optimal ply vs our policy

maybe for a mate in 5 or more

Future Works

* zobrist hashing and invariance properties in chess endgames to reduce state space cardinality

Why is it so difficult to solve chess endgames?

> *A player can sometimes afford the luxury of an inaccurate move, or even a definite error, in the opening or middlegame without necessarily obtaining a lost position. In the endgame ... an error can be decisive, and we are rarely presented with a second chance.*

- Paul Keres

Thank You!

Why $\gamma = 0.99$?

The longest rkr endgame requires 16 white moves, hence $\gamma^{16} \approx 0.85$ is a good reward for reaching the goal and producing fast mates.